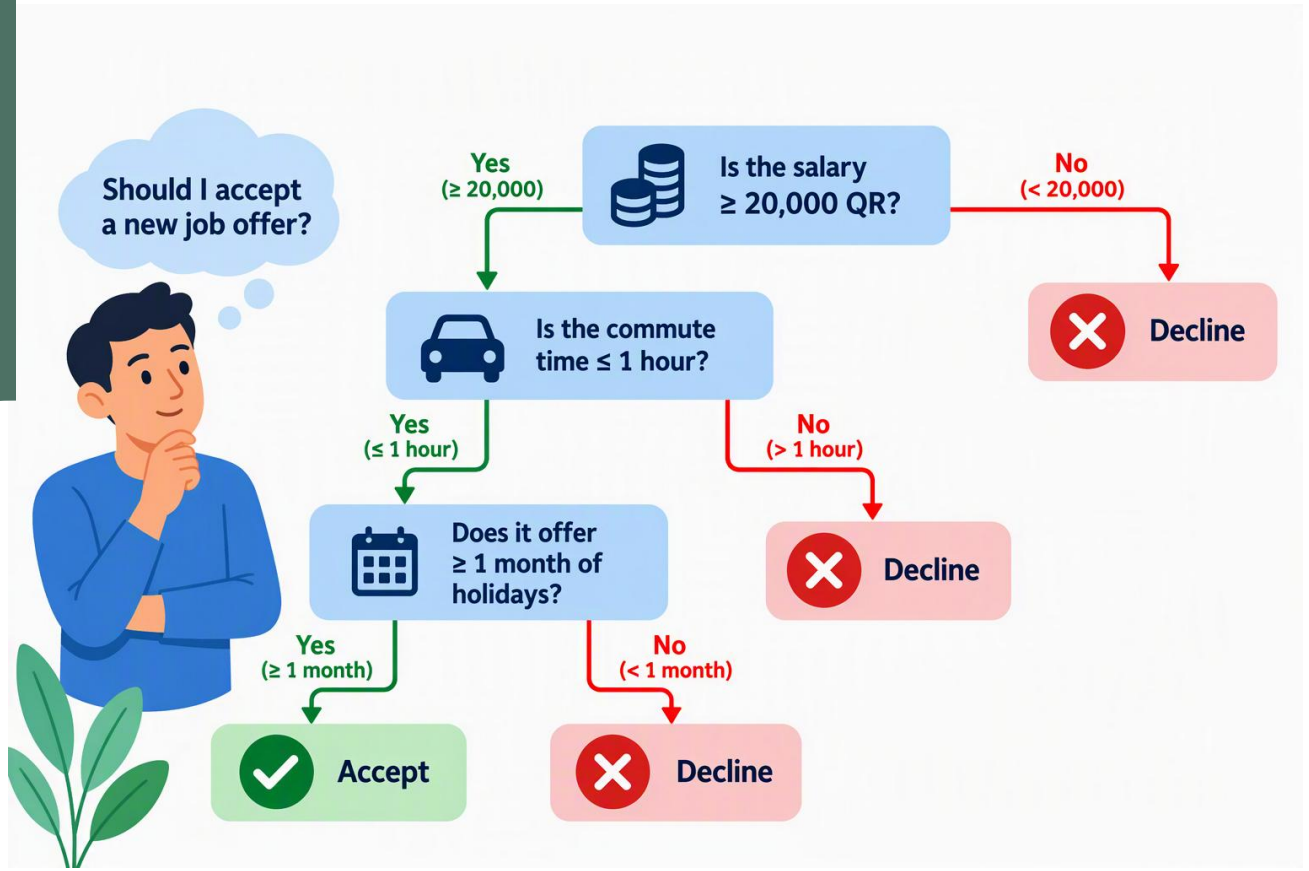
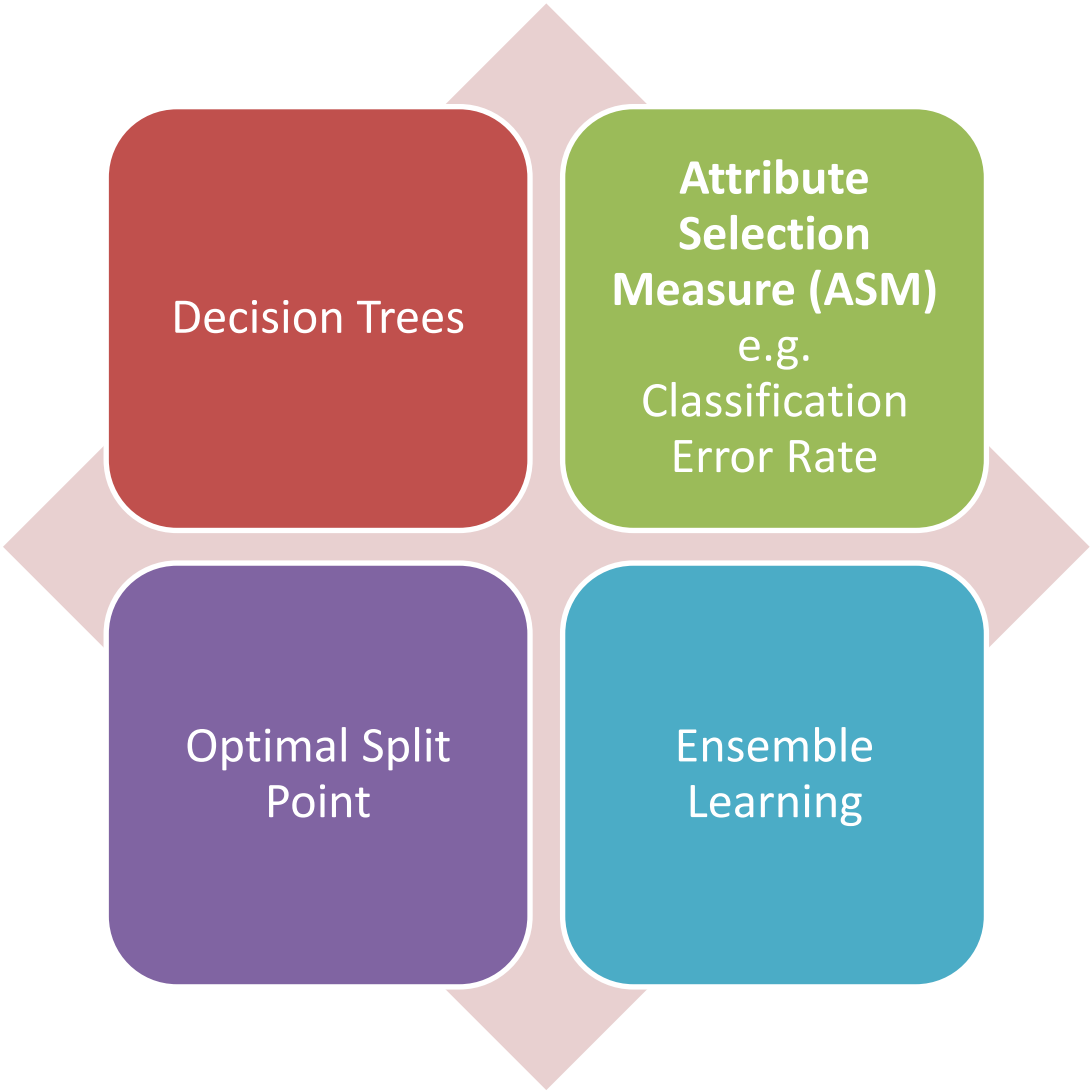
Classification using Decision Trees



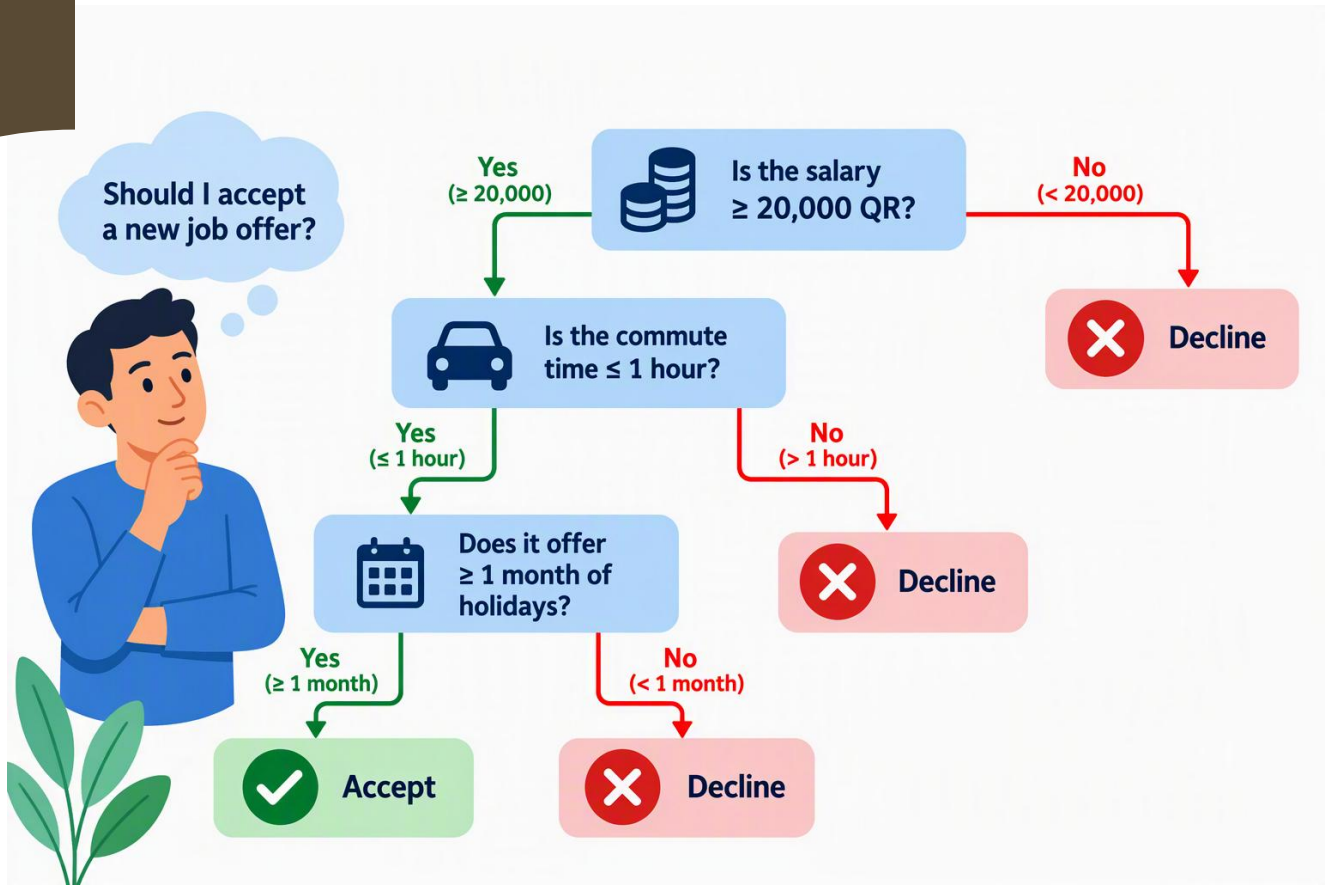
Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly Microsoft Copilot and Google Gemini

Outline

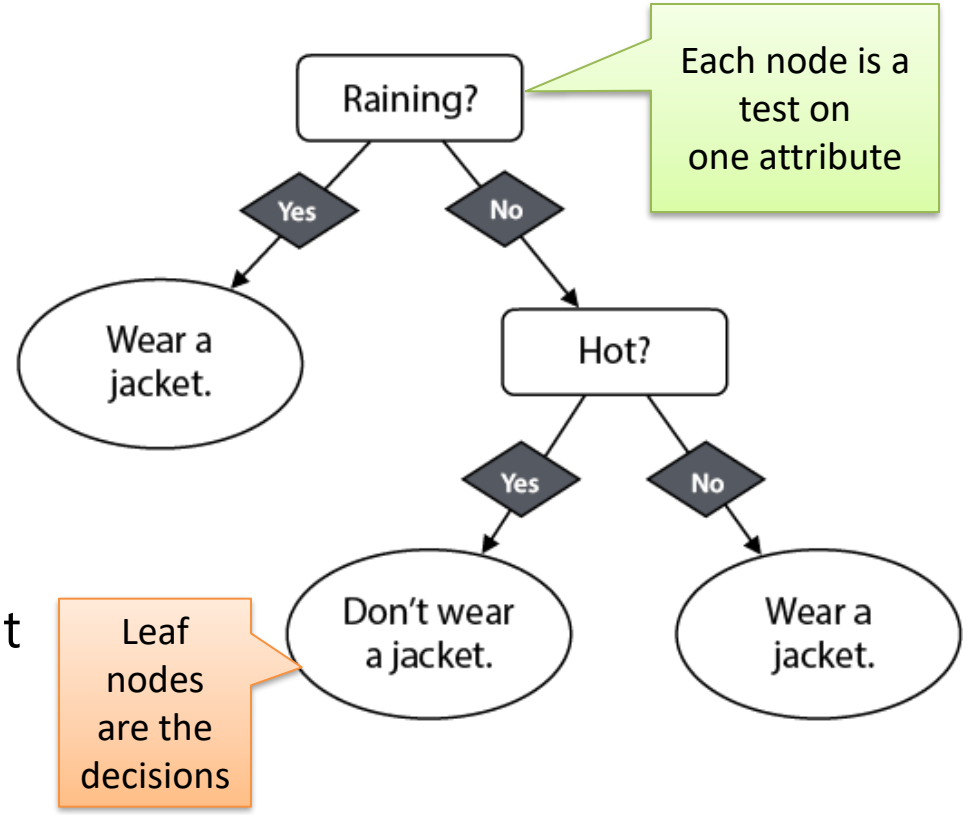

Decision Trees
Attribute Selection Measure (ASM) e.g. Classification Error Rate
Optimal Split Point
Ensemble Learning

Decision Trees



Should I wear jacket today?

- 🌧️ Raining → Wear a jacket 🧥
- 🌞 Not raining → Check temperature
  - 🔥 Hot → Don't wear a jacket
  - 🥶 Cold → Wear a jacket 🧥



Key ideas of a Decision Tree:

🌲 **DT**: depicts the decision process

❓ **Each node:** asks a question about one attribute

🌿 **Each branch:** outcome of the question (yes or no)

🎯 **Each leaf:** final decision/class

⬇️ **Flow:** top → bottom until reaching the decision

We arrive to a decision (i.e., a class) by asking a series of questions

Image source

Decision Tree (DT)

🌳 **DT**: is a supervised ML model that uses a sequence of yes-or-no questions to classify instances

- **Key components:**

🔝 **Root node:** starting point of the model

🔍 **Decision nodes:** test a feature

🔀 **Branches:** outcomes of tests

🎯 **Leaf nodes:** final class prediction

When to use DTs:

📘 When transparent **rule-based decisions** are required

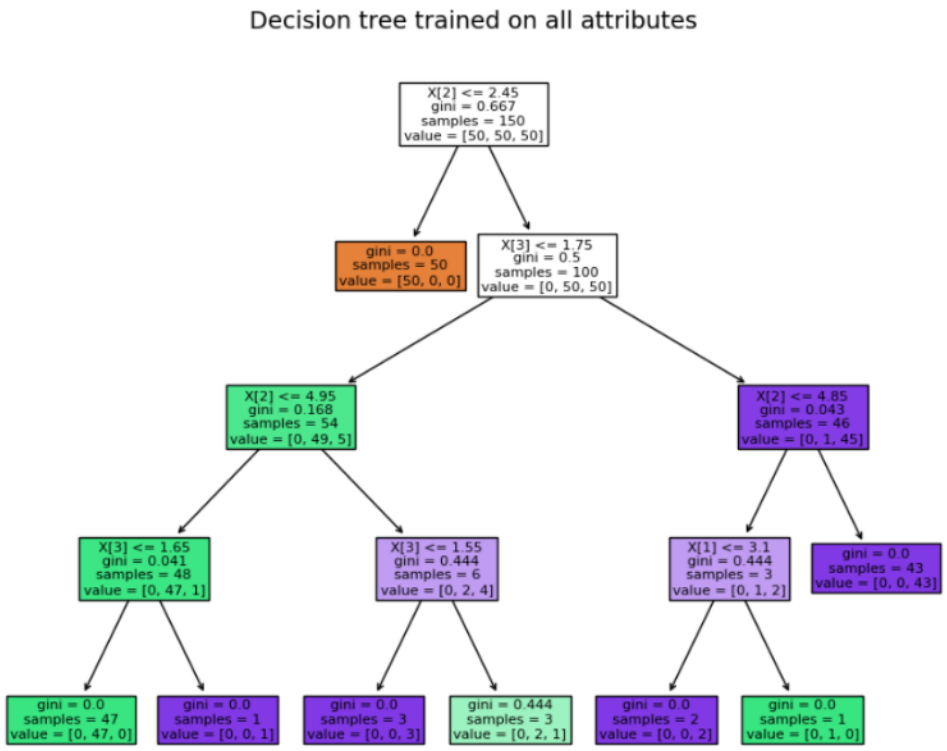
🧭 When decisions involve **conditional checks** across multiple attributes

👀 When **interpretability** and **explainability** are essential (e.g., loan approval, fraud detection, medical triage)

Decision Tree for the Iris dataset: using all the four attributes

```
X_iris = iris.data.values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=4, random_state=42)
tree_clf = tree_clf.fit(X_iris, y_iris)
```

```
tree_clf.fit(X_iris, y_iris)
plt.figure(figsize=(10,8))
plot_tree(tree_clf, filled=True)
plt.title("Decision tree trained on all attributes")
plt.show()
```



In [31]:
```
print(tree_clf.predict([[.5, 1.5,5,7.5]]))
print(tree_clf.predict([[.5, 1.5,2.5,1.2]]))
print(tree_clf.predict([[5, 1.5,2,3]]))
```

In [31]:
```
print(tree_clf.predict([[.5, 1.5,5,7.5]]))
print(tree_clf.predict([[.5, 1.5,2.5,1.2]]))
print(tree_clf.predict([[5, 1.5,2,3]]))
```

```
[2]
[1]
[0]
```



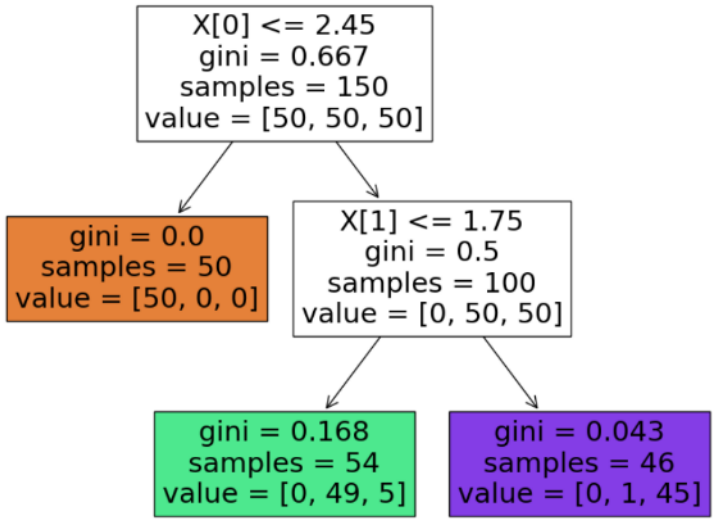
07.dt\1.dt_example.ipynb

Decision Tree for the Iris dataset: using two attributes

```
In [15]:
iris = load_iris(as_frame=True)
columns_to_use = ["petal length (cm)", "petal width (cm)"]
X_iris = iris.data[columns_to_use].values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf = tree_clf.fit(X_iris, y_iris)
```

```
In [18]:
plt.figure(figsize=(10,8))
plot_tree(tree_clf, filled=True)
plt.title("Decision tree trained on two attributes")
plt.show()
```

Decision tree trained on two attributes




```
In [19]:
tree_clf.predict([[3, 2.5]])
Out[19]: array([2])

In [16]:
tree_clf.predict([[5, 1.5]])
Out[16]: array([1])

In [17]:
tree_clf.predict([[.5, 1.5]])
Out[17]: array([0])
```

```
In [20]:
r = export_text(tree_clf, feature_names=columns_to_use)
print(r)
```

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2
```

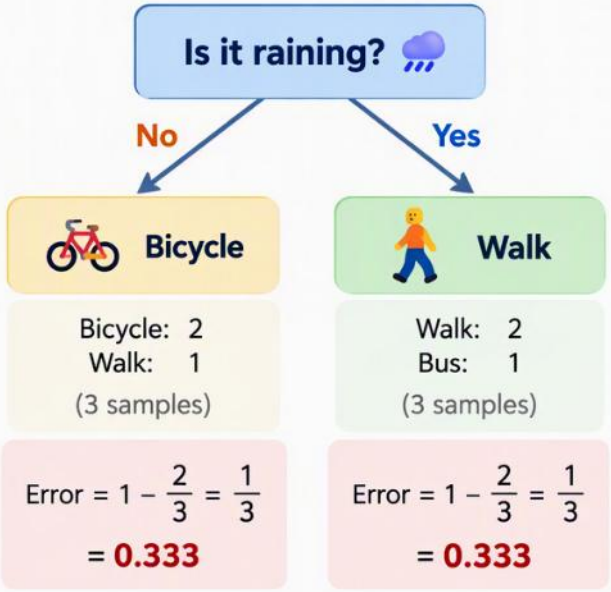


07.dt\5.decision_boundry_example.ipynb

Attribute Selection Measure (ASM) e.g., Classification Error Rate

Training Data (6 trips)

Distance	Raining?	Travel choice
0.5 km	Yes	Walk
1 km	No	Walk
1 km	Yes	Walk
3 km	No	Bicycle
5 km	No	Bicycle
5 km	Yes	Bus



Classification Error Rate

$$\text{Error} = 1 - \max(p_k)$$

Fraction of samples misclassified at a node.

Lower error indicates a purer node.

Weighted Classification Error for split on Raining?

$$\text{Error(Raining?)} = \frac{3}{6} \times \frac{1}{3} + \frac{3}{6} \times \frac{1}{3} = \frac{1}{3} = 0.333$$

How should I travel?

Predict Walk, Bicycle or Bus from distance and rain.

Distance	Raining?	Travel choice
0.5 km	Yes	Walk
1 km	No	Walk
1 km	Yes	Walk
3 km	No	Bicycle
5 km	No	Bicycle
5 km	Yes	Bus

Inputs: distance and rain

Target: travel choice



Walk



Bicycle



Bus

Candidate questions: distance ≤ 2 km, or raining?

Simplified dataset, not travel advice.

Classification error at a node

Each leaf predicts its most common class (the majority class).

Node error = wrong predictions / examples in the node

Actual class	Number of trips	If we predict Walk
Walk	3	Correct
Bicycle	2	Wrong
Bus	1	Wrong

Before splitting: 3 / 6 = 50% node error

Weighted Classification Error

Our Attribute Selection Measure (ASM) scores each candidate split.

Weighted error = Σ (child weight × child error)

For each child	Calculation	Meaning
Child weight	Child trips / Parent trips	Share of the data
Child error	Wrong / Child trips	Fraction wrong

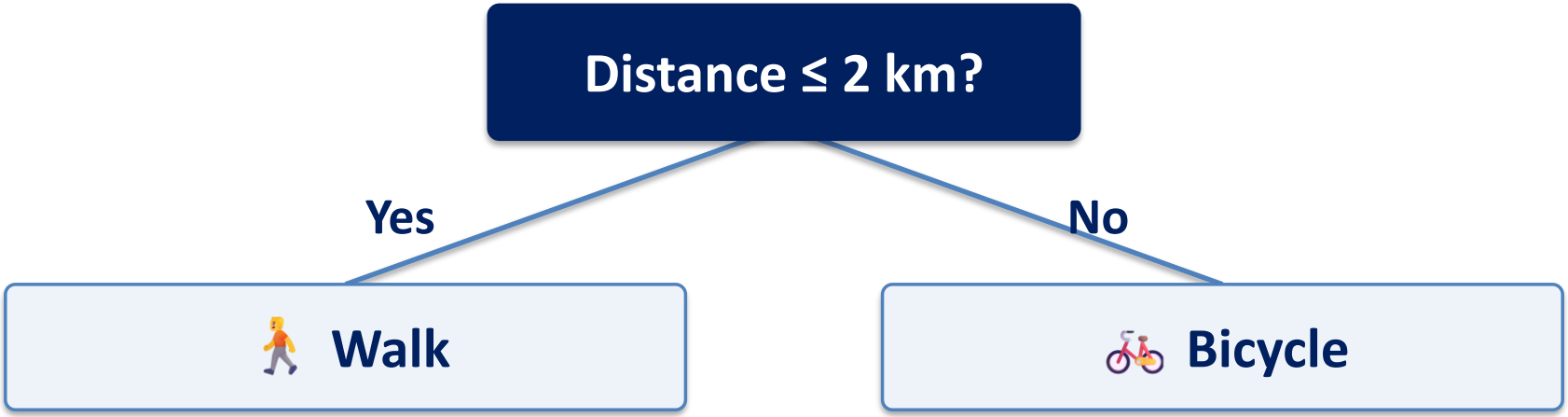
Σ means add across all children.

The candidate with the lowest weighted error wins.

Every trip counts equally. Larger children receive more weight.

Distance split

Each child contains three trips, so each has weight 3/6.



3 Walk trips

Node error = 0 / 3

2 Bicycle + 1 Bus

Node error = 1 / 3

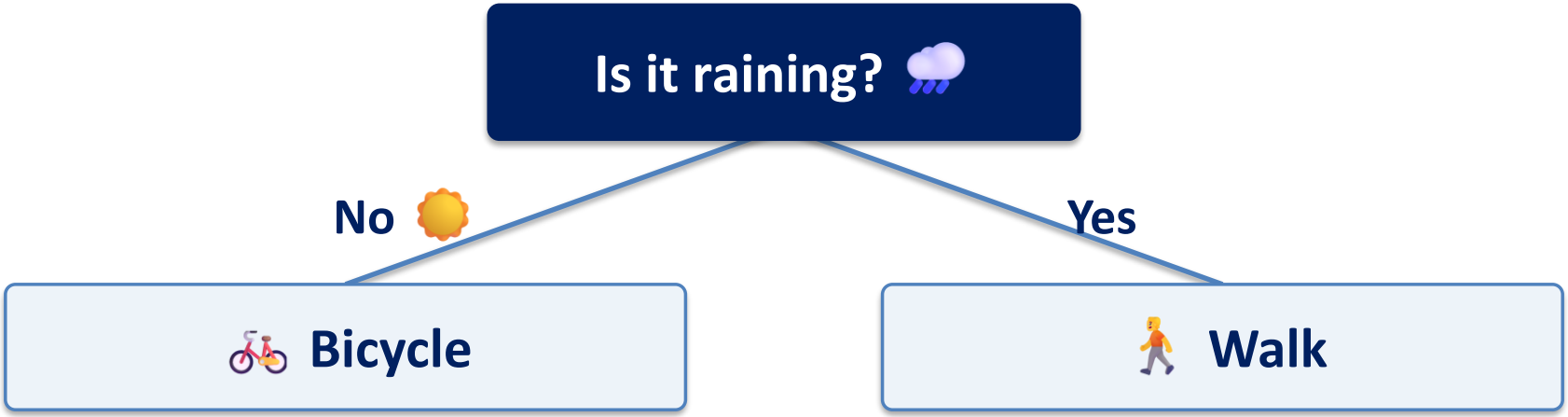
Weighted Classification Error

(3/6 × 0/3) + (3/6 × 1/3) = 16.7%

Only the 5 km rainy Bus trip is wrong.

Weather split

Use the same six trips and the same child weights.



2 Bicycle + 1 Walk

Node error = 1 / 3

2 Walk + 1 Bus

Node error = 1 / 3

Weighted Classification Error

(3/6 × 1/3) + (3/6 × 1/3) = 33.3%

Wrong trips: 1 km dry and 5 km rainy.

Distance wins the first split

Compare candidates on the same training examples.

Candidate split	Mistakes	Weighted error
Distance ≤ 2 km?	1 out of 6	16.7%
Raining?	2 out of 6	33.3%

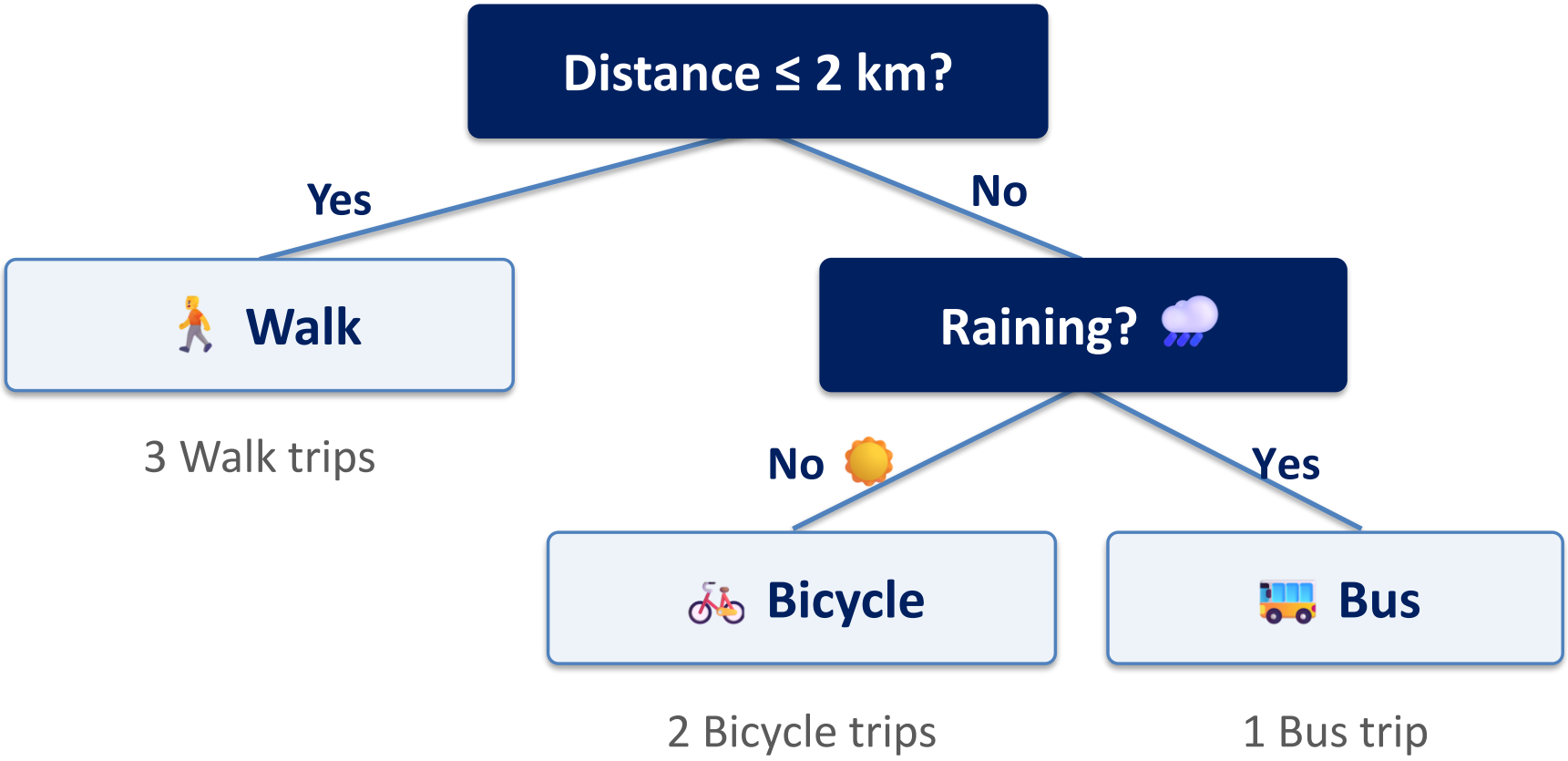
Lower weighted error wins

Distance becomes the root: 16.7% < 33.3%.

These scores measure training fit, not performance on new trips.

The complete travel tree

Only the longer trips need a second question about rain.



Final weighted leaf error = 0%

Perfect training predictions still need checking on new trips.

Travel predictions: answers

Each answer follows the decision tree on slide 16.

Trip	Distance	Raining?	Answer and reason
A	0.8 km	Yes	🚶 **Walk** Distance ≤ 2 km
B	4 km	No	🚲 **Bicycle** Distance > 2 km, no rain
C	4 km	Yes	🚌 **Bus** Distance > 2 km, rain

Trip A skips the rain question: distance already gives Walk.

For trips B and C, the rain question decides the class.

Other Attribute Selection Measures

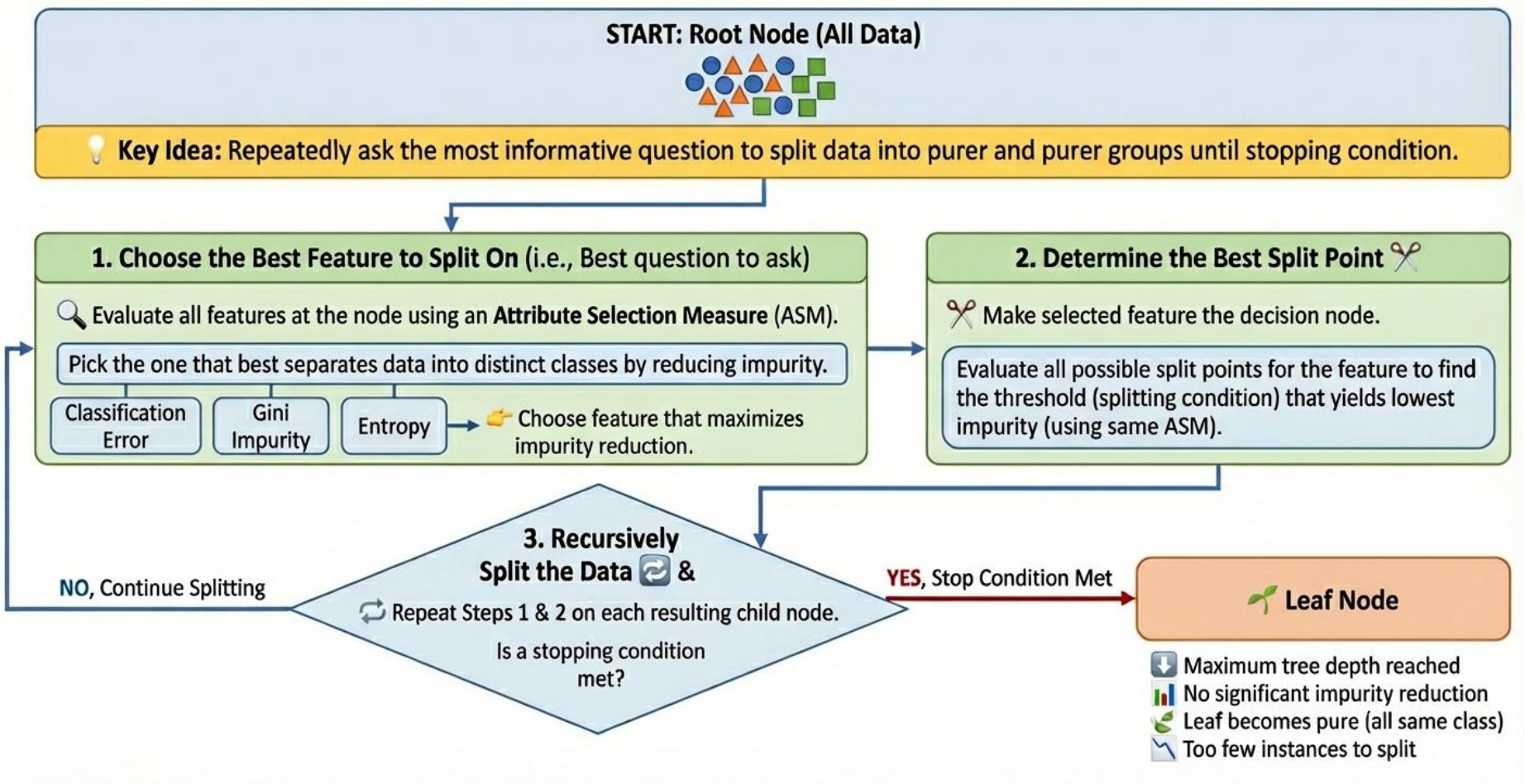
- **Gini Impurity**
  - Measures the probability of **incorrect classification** if labels are assigned according to the class distribution.

$$Gini(t) = 1 - \sum_{i=1}^{k} p_i(t)^2$$

Where $\boldsymbol{p_i(t)}$ = proportion of class $\boldsymbol{i}$ at node $\boldsymbol{t}$,
and $\boldsymbol{k}$ = number of classes at node $\boldsymbol{t}$

  - **0 = perfectly pure**; lower is better.
- **Entropy**
  - Measures the **uncertainty/disorder** in the class distribution.
  - $Entropy = -\sum p_k \log_2(p_k)$
  - **0 = perfectly pure**; lower is better.

Building a Decision Tree – Key Steps



DT constructs decision boundaries

- Decision Trees build **boundaries** by splitting the feature space
  - A DT repeatedly splits the feature space into **rectangular** regions (boxes) using simple rules to fit the linearly inseparable data
  - Each split adds a vertical or horizontal line in the 2D feature space

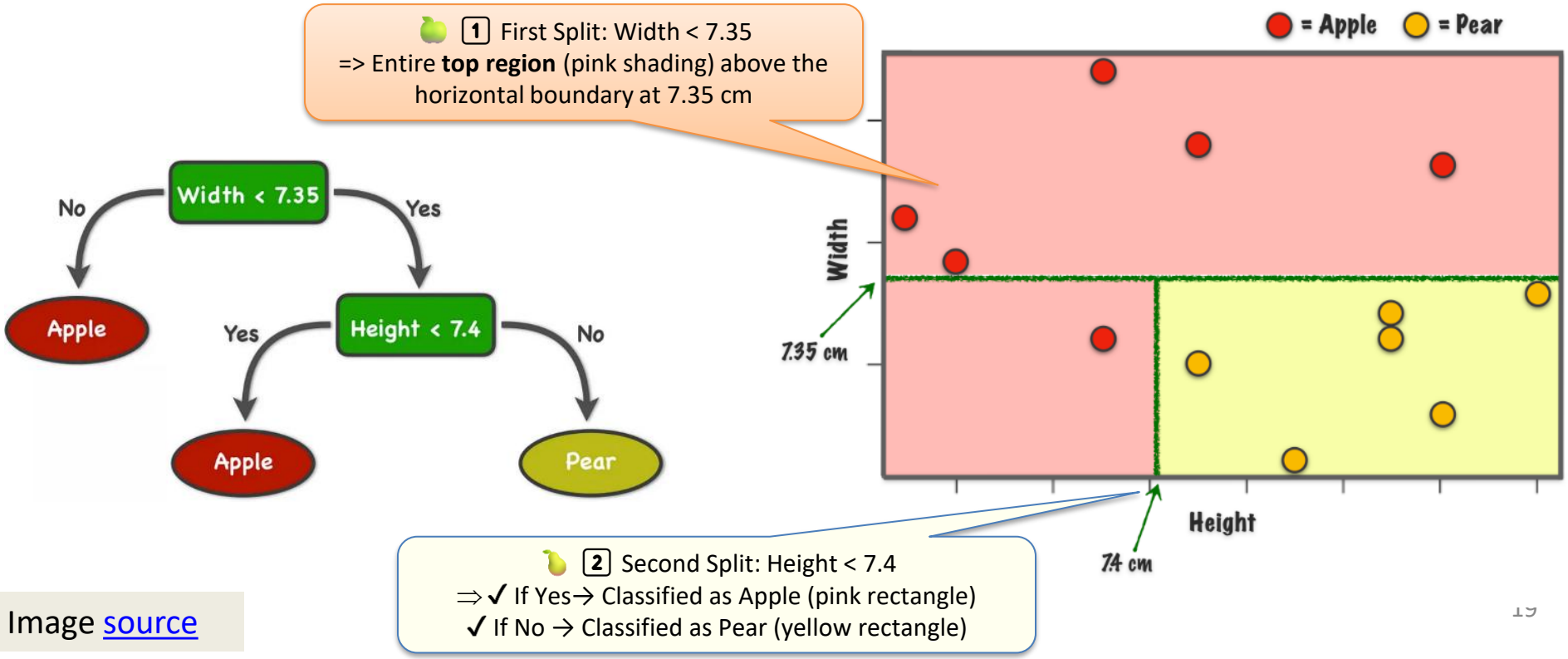
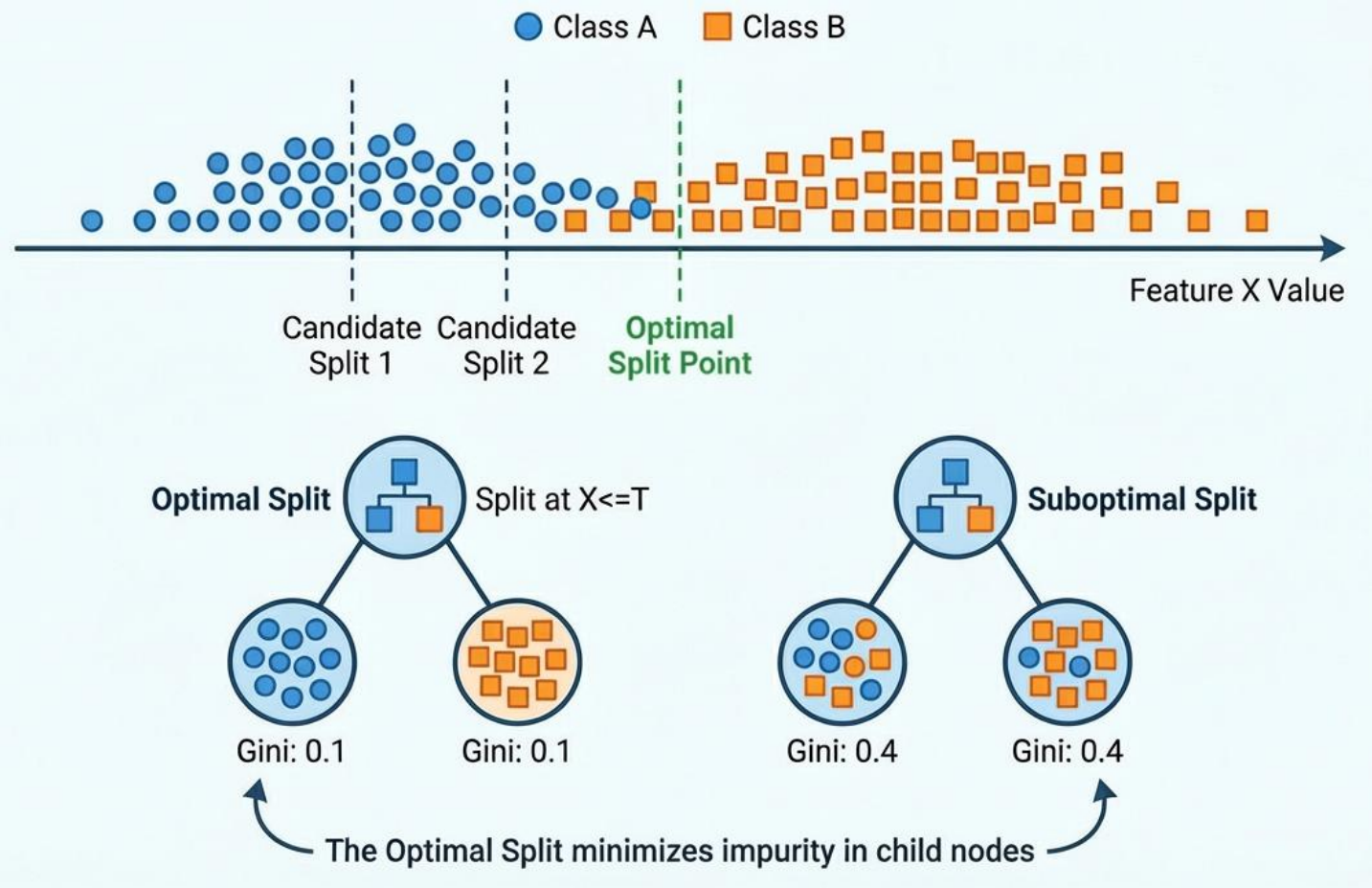


Image source

Optimal Split Point



Questions a tree can ask

A split separates trips using one question.

Input type	Travel example	Split question
Categories	Yes or No	Is it raining?
Ordered groups	Short, Medium, Long	Is the trip Short?
Numbers	Distance in km	Distance ≤ 2 km?

Split score: Weighted Classification Error

Lower score = fewer training mistakes.

Short, Medium and Long are optional distance groups.

Category example: rain

Each group predicts its most common travel choice.

Raining?	Trips in the group	Predict	Mistakes
No	2 Bicycle, 1 Walk	Bicycle	1
Yes	2 Walk, 1 Bus	Walk	1

Weighted error = 2 mistakes ÷ 6 trips

33.3%

Each trip counts equally in this training score.

Ordered groups: short to long

Short, Medium and Long have a natural order.

Group	Distance
Short	1 km or less
Medium	Over 1 km, up to 3 km
Long	Over 3 km

A single cutoff keeps the groups in order:

Short versus Medium + Long

Short + Medium versus Long

These optional groups come from the original distances.

Finding distance cutoffs

Sorted distances: 0.5, 1, 3, 5 km.

Try a cutoff halfway between each pair.

Neighboring values	Halfway cutoff
0.5 and 1 km	0.75 km
1 and 3 km	2 km
3 and 5 km	4 km

Two groups: at or below the cutoff, and above it.

A cutoff is also called a threshold.

Which distance cutoff works best?

Compare the mistakes after each split.

Distance cutoff	Mistakes	Weighted error
0.75 km	3 out of 6	50%
2 km	**1 out of 6**	**16.7%**
4 km	2 out of 6	33.3%

2 km makes the fewest mistakes.

Each group predicts its most common travel choice.

The winning distance split

Question: Is the distance 2 km or less?

Distance group	Predict	Mistakes
≤ 2 km: 3 Walk	Walk	0
> 2 km: 2 Bicycle, 1 Bus	Bicycle	1

Weighted error = 1 out of 6 = 16.7%

Decision tree hyperparameter tuning

Hyperparameters are settings we choose before training.
Tuning helps a tree perform well on unseen data.

Setting	What it controls
max_depth	Maximum tree depth. Smaller values limit complexity.
min_samples_split	Minimum examples needed to split a node.
min_samples_leaf	Minimum examples in each leaf. Larger values discourage tiny leaves.
criterion	How a tree scores splits: Gini impurity or entropy.

Too much complexity can overfit.

Too little can miss useful patterns.

The parameter grid

GridSearchCV tries every combination of these values.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import f1_score

# Values to try for each setting
param_grid = {
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'criterion': ['gini', 'entropy']
}
```

4 × 3 × 3 × 2 = 72 combinations

None means no explicit depth limit.

GridSearchCV with F1-score

Five-fold CV: train on four folds, validate on the remaining fold.

F1 helps when classes are imbalanced or precision and recall both matter.

```
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,           # Five validation folds
    scoring='f1',   # Balance precision and recall
    n_jobs=-1       # Use all available CPU cores
)
# Search using training data only
grid.fit(X_train, y_train)
print("Best parameters:", grid.best_params_)
print("Best CV F1-score:", grid.best_score_)
```

F1: harmonic mean of precision and recall.

The highest mean CV F1 selects the settings.

72 combinations × 5 folds = 360 CV fits.

Final test-set evaluation

The test set stays outside every tuning fold.

Continue with the grid fitted on the previous slide.

```
# GridSearchCV best selected model
best_model = grid.best_estimator_

# Evaluate on the untouched test set
y_pred = best_model.predict(X_test)
print("Test F1-score:", f1_score(y_test, y_pred))
```

CV F1 helps choose the settings.

Test F1 evaluates the selected model

on held-out examples.

Summary

- A **Decision Tree** recursively selects the most **informative feature** and the best **split point** at each step until a stopping criterion is reached 🌳

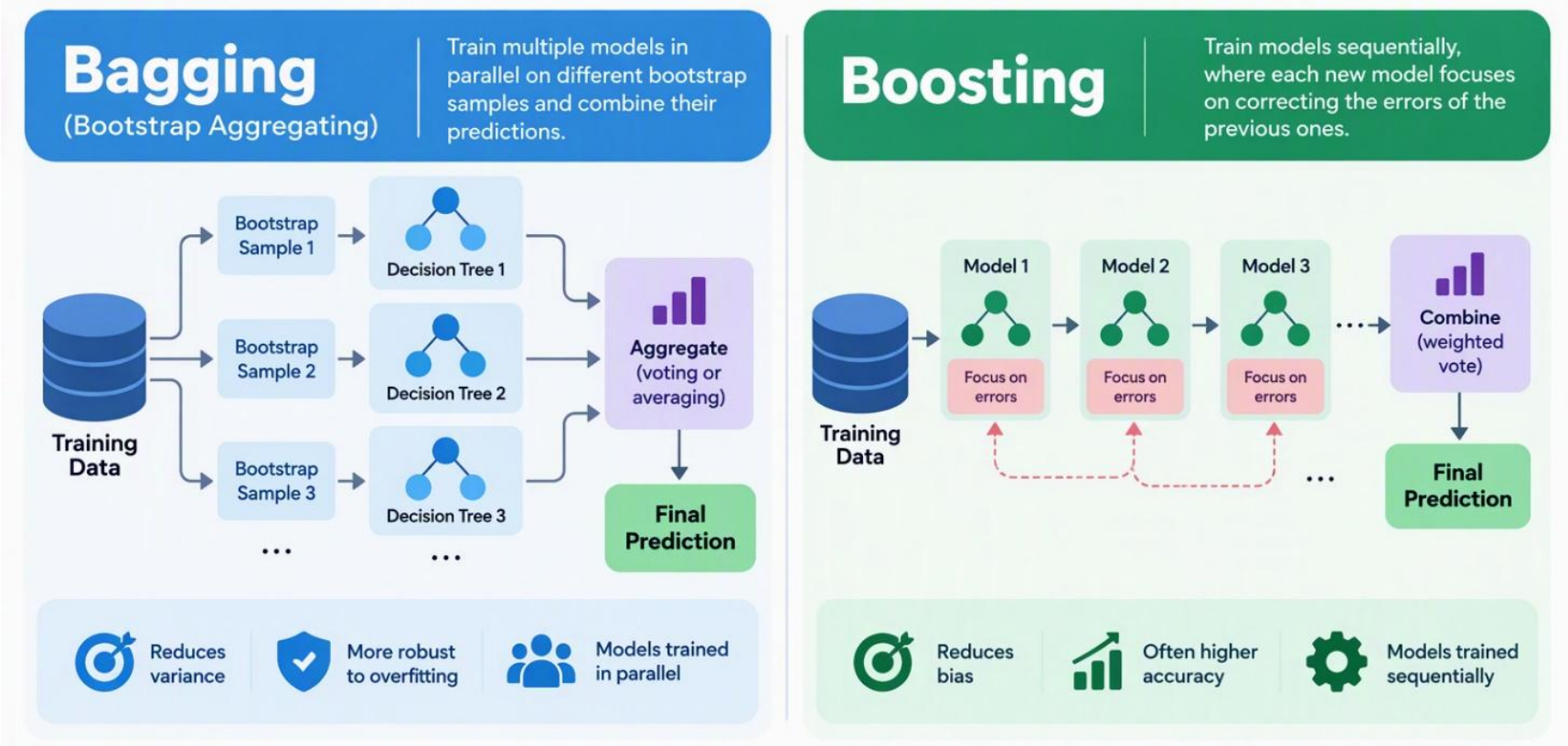
✅ **Pros**

- 🧠 **Interpretable** — easy to visualize and explain
- 🎯 Selects useful features during training
- 🔧 No feature scaling needed

⚠️ **Cons**

- 📏 **Greedy algorithm** — locally optimal decisions may miss a better overall tree
- ➖ **Limited expressiveness** — only horizontal/vertical boundaries, may struggle with complex structures
- 🌪️ **Unstable** — Small data changes can produce a different tree

Ensemble Learning

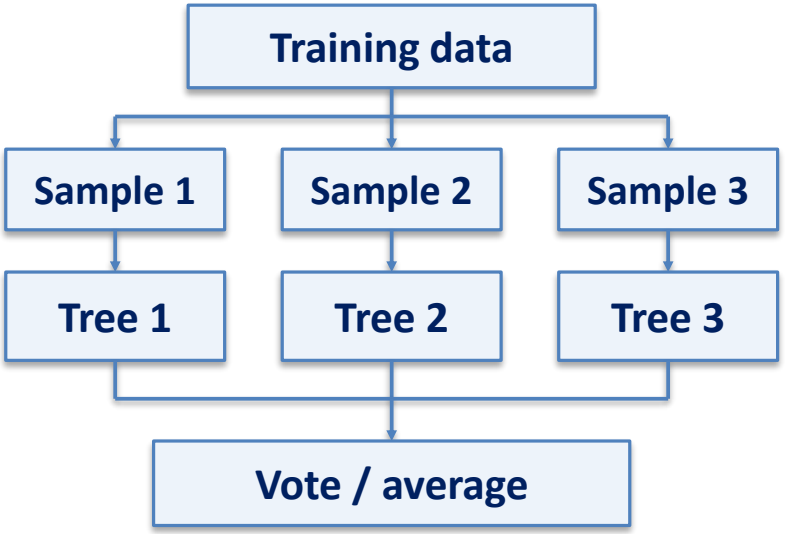


Bagging and Boosting

Ensemble learning combines several models into one prediction.

BAGGING

Train independently in parallel



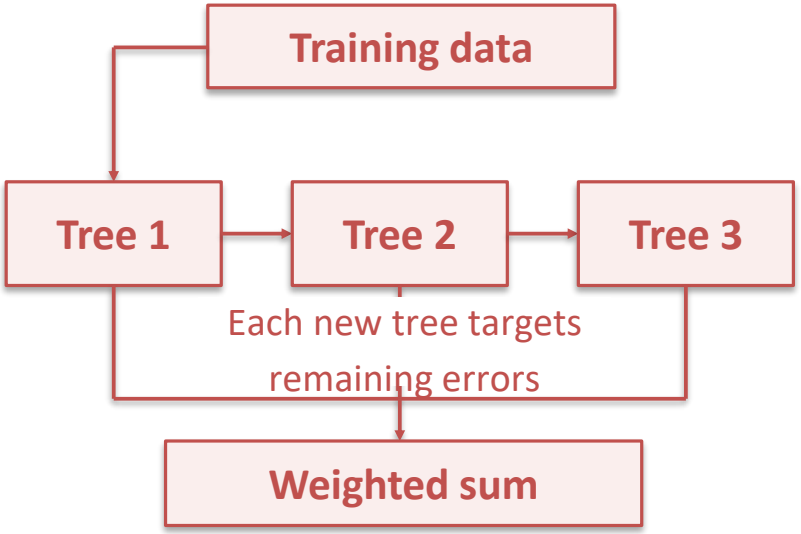
Use: reduce overfitting by combining predictions from multiple models

Combine: class votes or mean outputs

Example: Random Forest*

BOOSTING

Train sequentially to correct errors



Use: reduce bias in underfitting models

Combine: weighted learner outputs

Example: Gradient Boosted Trees

Bootstrap = sampling with replacement. * Random Forest also samples features at each split.

Bagging: independent trees

Different random samples help trees make different errors.

Combining their predictions mainly reduces variance.

Simple dataset	Inputs (X)	Target (y)
Iris: classification	4 flower measurements	One of 3 species
Diabetes: regression	10 baseline measurements	Progression score

1 Resample rows Sample training rows with replacement.

2 Fit independently Train one decision tree per sample, in parallel.

3 Combine Average class probabilities or numeric predictions.

Random forest adds random feature choices at each split to bagged trees.

Bagging for classification

Iris: predict flower species from four measurements.

X = sepal/petal lengths and widths. y = species. Accuracy: higher is better.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model = BaggingClassifier(
    n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 3))
```

Example run: Accuracy: 0.967 on 30 held-out flowers.

stratify=y preserves class proportions. Keep the 20% test set out of tuning.

Tuning bagging

Hyperparameters are settings chosen before fitting.

Within training data, validate on one fold and train on the other two.

Validate	Train	Train
Train	**Validate**	Train
Train	Train	**Validate**

3 rounds per setting. Average validation accuracy.

The original test set stays outside all folds.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

base = BaggingClassifier(
    estimator=DecisionTreeClassifier(), random_state=42)
grid = {"n_estimators": [50, 100],
        "estimator__min_samples_leaf": [1, 5]}
search = GridSearchCV(base, grid, cv=3, scoring="accuracy")
search.fit(X_train, y_train)
print(search.best_params_)
print("CV accuracy:", round(search.best_score_, 3))
print("Test:", round(search.score(X_test, y_test), 3))
```

Example: 50 trees, at least 5 samples per leaf. CV accuracy = 0.958.

n_estimators = tree count. estimator__min_samples_leaf = leaf size in each tree.

Bagging for regression

Diabetes: predict a progression score from ten measurements.

X = baseline measurements. y = numeric score. MAE: lower is better.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingRegressor
from sklearn.metrics import mean_absolute_error

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = BaggingRegressor(
    n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("MAE:", round(mean_absolute_error(y_test, pred), 3))
```

Example run: MAE: 44.441 progression-score units.

MAE = average absolute error. Educational example, not clinical use.

Boosting: small sequential corrections

Each new tree aims to improve the current ensemble.

Shallow trees often work well as small corrections.

Method	What the next model focuses on
AdaBoost	Gives more weight to difficult examples.
Gradient boosting	Fits corrections that reduce a chosen loss.

A regression example

Actual score = 200. Current prediction = 150. Residual = 50.

New tree predicts a correction of 40, with learning_rate = 0.1.

Updated prediction = 150 + 0.1 × 40 = 154

Tune learning_rate, n_estimators and max_depth together.

Smaller learning rates usually need more stages.

Boosting for classification

Iris: predict flower species from four measurements.

X = sepal/petal lengths and widths. y = species. Accuracy: higher is better.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model = GradientBoostingClassifier(
    n_estimators=100, learning_rate=0.05,
    max_depth=2, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 3))
```

Example run: Accuracy: 0.933 on 30 held-out flowers.

stratify=y preserves class proportions. Keep the 20% test set out of tuning.

Boosting for regression

Diabetes: predict a progression score from ten measurements.

X = baseline measurements. y = numeric score. MAE: lower is better.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = GradientBoostingRegressor(
    n_estimators=100, learning_rate=0.05,
    max_depth=2, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("MAE:", round(mean_absolute_error(y_test, pred), 3))
```

Example run: MAE: 42.546 progression-score units.

MAE = average absolute error. Educational example, not clinical use.

Tuning boosting

Try 4 settings with 3-fold cross-validation.

Choose the highest average R^2, then evaluate on the test set.

n_estimators = number of stages. max_depth = complexity of each tree.

```
from sklearn.model_selection import GridSearchCV

model = GradientBoostingRegressor(random_state=42)
grid = {"n_estimators": [50, 100], "max_depth": [1, 2]}
search = GridSearchCV(model, grid, cv=3, scoring="r2")
search.fit(X_train, y_train)

print("Best settings:", search.best_params_)
print("CV R²:", round(search.best_score_, 2))
print("Test R²:", round(search.score(X_test, y_test), 2))
```

Example run: depth 1, 100 stages. CV R^2 = 0.45. Test R^2 = 0.48.

R^2: higher is better. 1 = perfect predictions.

0 = predicting the evaluation-set mean. Negative = worse than that baseline.

Summary

- **Decision trees:** predict through feature-based questions, ending at a leaf.
  - **Split selection:** compares features and thresholds using error or impurity.
- **Tuning:** controls tree complexity. GridSearchCV selects settings with cross-validation.
- **Ensembles:** bagging combines independent trees to reduce overfitting. Boosting adds trees sequentially to correct errors and reduce underfitting.
- **Evaluation:** keeps test data separate. F1 balances precision and recall. MAE and R^2 assess regression.

Resources

Decision trees

Splits, Gini, entropy, tree depth and overfitting.
https://scikit-learn.org/stable/modules/tree.html

Bagging and boosting

Ensemble concepts with classification and regression examples.
https://scikit-learn.org/stable/modules/ensemble.html

GridSearchCV and cross-validation

Parameter grids, scoring and model selection.
https://scikit-learn.org/stable/modules/grid_search.html

Evaluation metrics

Precision, recall, F1-score, MAE and R^2.
https://scikit-learn.org/stable/modules/model_evaluation.html